

Hooking Swift (The Hard Way)

Francesco Tamagni (mrmacete)
Håvard Sørbo (hsorbo)



MAScon Berlin
October 2026



The Hard Way

- avoid bridges, treat compiled Swift as “just” native code
 - ignore complexity of Swift as a whole
- design ad-hoc hooks on specific target functions
 - look at the compiled code
 - stick to exported symbols
 - hooks apply to all apps for those functions
- simple reusable techniques

Hooking Targets

- simple test app with CryptoKit
- hooking targets
 - `SymmetricKey()` constructor
 - dump the constructed key data
 - `HKDF<HashFunction>.deriveKey()`
 - dump input / output key data
 - dump the hash algorithm name

Swift Calling Convention (arm64)

<https://github.com/swiftlang/swift/blob/main/docs/ABI/CallingConventionSummary.rst#arm64>

<https://github.com/swiftlang/swift/blob/main/docs/ABI/CallingConvention.rst#function-signature-lowering>

Function arguments	x0 ... x7, then stack
Function return value	x0, x1, ...
self pointer	x20
output data pointer	x8
error pointer	x21 (if the function throws)
async context	x22 (if the function is async)

NOTE: this is a simplistic view, there's more to it

Convention Translation

```
const wrapperFunction = new NativeFunction(  
  trampoline,  
  retType,  
  [...argTypes, "...",  
    "pointer", // self  
    "pointer", // target function  
    "pointer", // error pointer  
    "pointer"  // output pointer  
]);
```

```
ldp x20, SCRATCH, [sp]  
ldp x21, x8, [sp, #0x10]  
br  SCRATCH
```

Target Swift
Function

Generics and Function Arguments

```
HKDF<SHA256>.deriveKey(  
    inputKeyMaterial: SymmetricKey,  
    outputByteCount: Int  
) -> SymmetricKey
```

args[0]	inputKeyMaterial
args[1]	outputByteCount
args[2]	type metadata for SHA256
args[3]	witness table for SHA256

Generics and Function Arguments

```
HKDF<SHA256>.deriveKey<Info>(
  inputKeyMaterial: SymmetricKey,
  info: Info,
  outputByteCount: Int
) -> SymmetricKey where Info :
DataProtocol
```

args[0]	inputKeyMaterial
args[1]	info
args[2]	outputByteCount
args[3]	type metadata for SHA256
args[4]	type metadata for DataProtocol instance
args[5]	witness table for SHA256
args[6]	witness table for DataProtocol instance

Type Name From Witness Table

1 [protocol witness table](#)

ProtocolConformanceDescriptor*
...

2 [protocol conformance descriptor](#)

Offset	Field
0	Protocol
4	TypeRef
8	WitnessTablePattern
12	ConformanceFlags

[DirectTypeDescriptor](#)
[IndirectTypeDescriptor](#)

3

[TypeReferenceKind](#)

4a

[type context descriptor](#) and [parent](#)

Offset	Field
0	Flags
4	Parent
8	Name

4b

[IndirectObjCClass](#)

ObjC class

[DirectObjCClassName](#)

ObjC class name (string)

4c



Thank you!
Questions?



MAScon Berlin
October 2026